BANK MANAGEMENT SYSTEM

AUTHOR: S. NISANTH / Employee ID: 12576

Question:

This program simulates a bank account management system. It utilizes an Account struct to organize account information, including the account number, holder's name, balance, and a history of up to ten transactions. A Bank class encapsulates the core logic for managing multiple accounts, stored in an array. The program begins with a login prompt, requiring the username "admin" and password "12345" for access to account management functions. Upon successful login, a menu-driven interface appears, offering several options.

Code:

```
#include <iostream>
#include <array>
#include <algorithm>

using namespace std;

const int MAX_SIZE = 100;

struct Account
{
   int accNo;
   string name;
   double balance;
};

class Bank
{
public:
```

```
//Login Module:
bool login()
{
   string username;
   int password;

   cout << "Enter username: ";
   cin >> username;
   cout << "Enter passowrd: ";
   cin >> password;

   if (username == "admin" && password == 12345)
   {
      return true;
   }
   else
   {
      return false;
   }
}

//Funtion to create a new record at the end of the existing list/array:
void CreateNewAccount(Account accounts[], int &n)
{

   if (n == MAX_SIZE)
   {
      cout << "\nThe Database is full!!" << endl;
      return;
```

```
    }
    else
    {
        n++;

        cout << "\nEnter account number: ";
        cin >> accounts[n].accNo;
        cout << "Enter account holder's name: ";
        cin >> accounts[n].name;
        cout << "Enter balance (in dollars): ";
        cin >> accounts[n].balance;
    }
}

//Function to search and display a specific account with the account number alone:
void DisplayAccountDetails(Account accounts[], int &n)
{
    //To check if array is empty:
    if (n < 0)
    {
        cout << "\nThe Database is empty!!" << endl;
        return;
    }

    int target;

target1:
    cout << "\nEnter the account number to display: ";
    cin >> target;
```

```
    int i;
    bool found = false;
    for (i = 0; i <= n; i++)
    {
        if (accounts[i].accNo == target)
        {
            found = true;
            break;
        }
    }

    if (found)
    {
        cout << "\nThe Requested Account: ";
        cout << "\n--------------";
        cout << "\nAccount Number: " << accounts[i].accNo << "\nAccount Holder Name: " << accounts[i].name << "\nBalance: $" << accounts[i].balance;
        cout << "\n--------------"<< endl;
    }
    else
    {
        cout << "\nThe Account you are searching for does not exist." << endl;
    }
}

//Function to display all accounts:
void DisplayAllAccounts(Account accounts[], int &n)
{
    //To check if array is empty:
```

```
        if (n < 0)
        {
            cout << "\nThe Database is empty!!" << endl;
            return;
        }

        cout << "\nThe Requested Accounts: ";
        cout << "\n----------";
        for (int i = 0; i <= n; i++)
        {
            cout << "\nAccount Number: " << accounts[i].accNo << "\nAccount Holder Name: " <<
accounts[i].name << "\nBalance: $" << accounts[i].balance;
            cout << "\n----------";
        }
        cout<< endl;
    }
};

int main()
{
    Account accounts[MAX_SIZE];
    int n = -1; //Used in place of index to track locations of active data.

    Bank bank; //Referencing Bank class?

start:

    cout << "\nBank Management System:";
    cout << "\n1.Login";
    cout << "\n2.Exit";
```

```
    int choice1;

checkpoint0:

    cout << "\nEnter your choice: ";
    cin >> choice1;

    switch (choice1)
    {
    case 1:
    {
        bool iflogin = bank.login();
        if (iflogin)
        {
            cout << "\nLogging In!!" << endl;
            goto checkpoint1;
        }
        else
        {
            cout << "\nInvalid Username or Password!! Try Again."<< endl;
            goto start;
        }
        break;
    }

    case 2:
        cout << "\nLogging Out!!" << endl;
        return 0;
```

```
    default:
    {
        cout << "\nInvalid Input!! Try Again.\n"
             << endl;
        goto checkpoint0;
        break;
    }
    }

checkpoint1:

    cout << "\nBank Management System Menu:";
    cout << "\n1.Create New Account.";
    cout << "\n2.Display a Specific Account Details.";
    cout << "\n3.Display All Account Details.";
    cout << "\n7.Exit.";
    int choice2;

choice2:

    cout << "\nEnter your choice: ";
    cin >> choice2;

    switch (choice2)
    {
    case 1:
        bank.CreateNewAccount(accounts, n);
        goto checkpoint1;
        break;
```

```
    case 2:
        bank.DisplayAccountDetails(accounts, n);
        goto checkpoint1;
        break;

    case 3:
        bank.DisplayAllAccounts(accounts, n);
        goto checkpoint1;
        break;

    case 7:
        cout << "\nLogging Out!!" << endl;
        return 0;

    default:
    {
        cout << "\nInvalid Input!! Try Again." << endl;
        goto checkpoint1;
        break;
    }
    }
}
```

Sample Outputs:

- **Scenario: If all inputs are correct:**

Exiting:

```
Bank Management System:
1.Login
2.Exit
Enter your choice: 2

Logging Out!!
```

Logging In:

```
Bank Management System:
1.Login
2.Exit
Enter your choice: 1
Enter username: admin
Enter passowrd: 12345

Logging In!!
```

Creating / Adding a new account:

```
Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: 1

Enter account number: 100
Enter account holder's name: Ben
Enter balance (in dollars): 1000

Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice:
```

Displaying a specific account:

```
Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: 2

Enter the account number to display: 100

The Requested Account:
--------------
Account Number: 100
Account Holder Name: Ben
Balance: $1000
```

Displaying All Accounts:

```
Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: 3

The Requested Accounts:
--------------
Account Number: 100
Account Holder Name: Ben
Balance: $1000
--------------
Account Number: 101
Account Holder Name: Charles
Balance: $1.21345e+10
--------------

Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice:
```

Exiting:

```
Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: 7

Logging Out!!
PS C:\Users\nisanth.saravanan\Desktop\L0 Assesment>
```

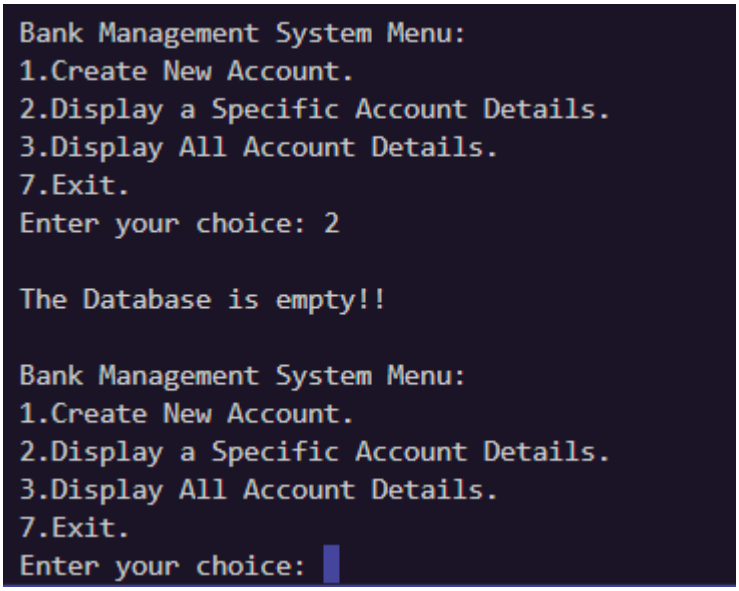
- **Scenario: If all inputs are incorrect:**

```
Bank Management System:
1.Login
2.Exit
Enter your choice: 5

Invalid Input!! Try Again.


Enter your choice:
```

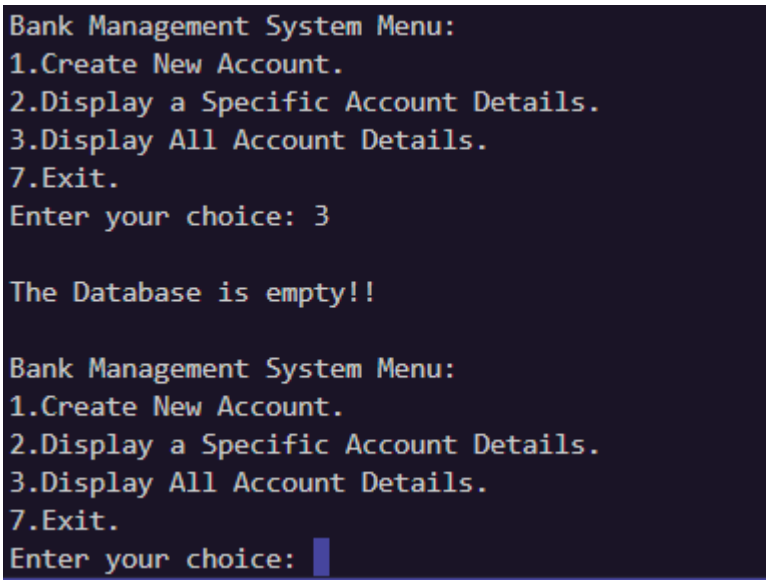
```
Enter your choice: 1
Enter username: asdaf
Enter passowrd: 1245

Invalid Username or Password!! Try Again.

Bank Management System:
1.Login
2.Exit
Enter your choice:
```

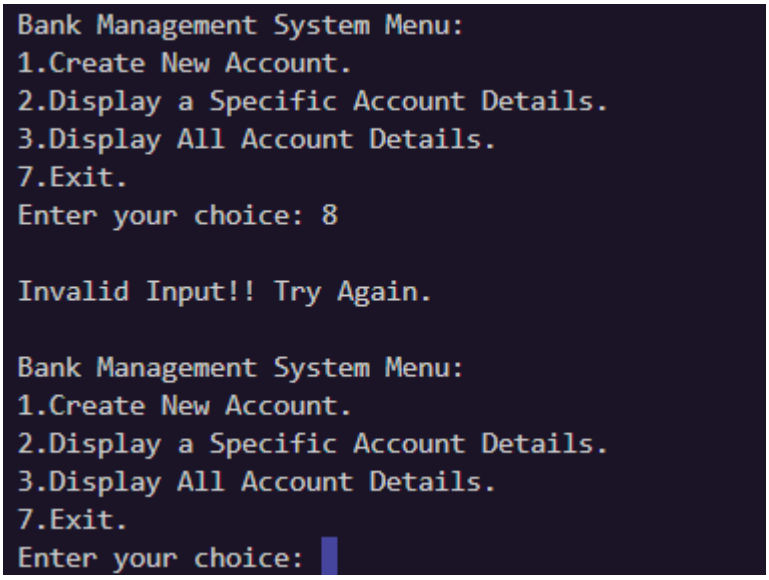
Performing on an empty record:

(Specific Search:)



(Display all records:)




Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: 8
Invalid Input!! Try Again.
Bank Management System Menu:
1.Create New Account.
2.Display a Specific Account Details.
3.Display All Account Details.
7.Exit.
Enter your choice: